

Вопросы к экзамену по дисциплине «Математическое и имитационное моделирование»

Содержание

1. Отличия математического и имитационного моделирования от других методов исследования социально-экономических систем	2
2. Наиболее часто используемые законы распределения в имитационном моделировании	2
3. Табличный, физический и программный способы получения случайных чисел	3
4. Псевдослучайные числа и методы их получения	3
5. Метод обратной функции: идея, пример, дискретный случай	4
6. Достоинства и недостатки метода обратной функции. Усечённое распределение	5
7. Метод композиции для сложных законов распределения	5
8. Метод принятия-отклонения	6
9. Способы генерирования нормального закона распределения	7
10. Тестирование генераторов случайных чисел на равномерность заполнения многомерного пространства	8
11. Тестирование генераторов случайных чисел на независимость	9
12. Построение моделей систем массового обслуживания. Примеры и части СМО	10
13. Основные критерии оценки работы СМО	11
14. Моделирование поступления заявок на основе стационарного пуассоновского процесса	12
15. Моделирование нестационарного пуассоновского процесса	13
16. Отличие непрерывных моделей от дискретных. Способы решения непрерывных моделей	14
17. Особенности непрерывных моделей. Примеры	16
18. Особенности моделей системной динамики Дж. Форрестера	17
19. Особенность агентно-ориентированных моделей и классификация среды . .	18
20. Особенность агентно-ориентированных моделей и классификация агентов .	19
21. Основная особенность агентно-ориентированных моделей, примеры	20
22. Способы продвижения модельного времени	21
23. Этапы исследования систем с помощью имитационного моделирования . . .	22
24. Какие модели называются адекватными? Валидация и верификация	23
25. Схема сравнения выходных данных реальной системы и модели	24
26. Планирование экспериментов на имитационных моделях. Факторы и отклики	25
27. Проблема сравнения альтернативных конфигураций. Метод общих случайных чисел	26
28. Проблемы синхронизации случайных чисел при сравнении альтернатив . . .	27
29. Алгоритм выбора лучшей конфигурации из множества альтернатив	28
30. Основная идея метода Монте-Карло. Примеры	30

Ниже — готовые ответы на первые 10 вопросов, в формате для устного экзамена.

1. Отличия математического и имитационного моделирования от других методов исследования социально-экономических систем

Математическое моделирование описывает систему с помощью формул, уравнений, функций, ограничений и зависимостей. Оно позволяет не просто наблюдать систему, а формально описать связи между её элементами: например, спрос, очередь, прибыль, затраты, поток клиентов.

Имитационное моделирование — это компьютерное воспроизведение работы системы во времени. Оно особенно полезно, когда система сложная, случайная, динамическая и её трудно описать одной простой формулой. В имитационной модели можно запускать эксперименты: менять параметры, смотреть последствия, проверять разные сценарии без вмешательства в реальную систему. В учебниках по simulation input analysis подчёркивается, что имитационные модели используют случайные входы: время прихода, время обслуживания, спрос, поломки, партии товаров и т. д. (textbook.simio.com)

Главное отличие от обычного наблюдения или статистики: моделирование позволяет не только анализировать прошлые данные, но и проверять сценарии “что будет, если...”. Например: что будет, если увеличить число кассиров, изменить интенсивность потока клиентов или добавить приоритет VIP-клиентов.

Главное отличие от реального эксперимента: эксперимент проводится не над реальной системой, а над моделью. Это дешевле, безопаснее и быстрее, особенно если реальные эксперименты дорогие, рискованные или невозможные.

2. Наиболее часто используемые законы распределения в имитационном моделировании

В имитационном моделировании часто задаются случайные величины: время между поступлениями заявок, время обслуживания, длительность ремонта, спрос, размер партии, число клиентов, вероятность отказа. Среди распространённых непрерывных распределений называют нормальное, экспоненциальное, равномерное и треугольное; среди дискретных — дискретное равномерное, биномиальное, Пуассона, геометрическое и отрицательное биномиальное. (textbook.simio.com)

Распределение	Что обычно задаёт
Равномерное $U(a, b)$	случайное значение в заданном диапазоне, если нет предпочтений
Экспоненциальное Пуассона	время между событиями: приход клиентов, заявки, поломки число событий за интервал времени
Нормальное $N(\mu, \sigma^2)$	естественные колебания вокруг среднего: спрос, время выполнения
Треугольное	время операции, если известны минимум, максимум и наиболее вероятное значение
Логнормальное	положительные величины с длинным “хвостом”: доходы, длительности, размеры заказов

Распределение	Что обычно задаёт
Вейбулла	время до отказа, надёжность оборудования
Эмпирическое	распределение по реальным наблюдениям

Для потока заявок часто используют **пуассоновский процесс**: число прибытий за интервал $([a,b])$ имеет распределение Пуассона со средним $\int_a^b \lambda(t) dt$, а при постоянной интенсивности межприходные интервалы имеют экспоненциальное распределение со средним $1/\lambda$. (textbook.simio.com)

3. Табличный, физический и программный способы получения случайных чисел

Табличный способ — заранее подготовленные таблицы случайных чисел. Плюсы: простота, можно использовать без компьютера. Минусы: ограниченный объём, неудобно для больших моделей, трудно автоматизировать.

Физический способ — числа получают из реального случайного физического процесса: шум, радиоактивный распад, тепловые колебания, квантовые процессы. Плюсы: ближе к “настоящей” случайности. Минусы: нужны устройства, возможны смещения, сложнее воспроизводить эксперимент.

Программный способ — числа создаёт алгоритм. Обычно это не настоящие, а **псевдослучайные** числа. Плюсы: быстро, удобно, можно воспроизвести результат по начальному значению — seed. Минусы: последовательность детерминирована, имеет период и может содержать скрытые закономерности.

В имитационном моделировании обычно нужны числа, похожие на независимые $U(0, 1)$, то есть равномерные на интервале от 0 до 1. Практически их получают алгоритмами, которые должны проходить тесты на равномерность и независимость. (textbook.simio.com)

4. Псевдослучайные числа и методы их получения

Псевдослучайные числа — это числа, полученные детерминированным алгоритмом, но выглядящие как случайные. NIST определяет PRNG как детерминированный вычислительный процесс, который использует входы, называемые seed, и выдаёт последовательность значений, выглядящую случайной по статистическим тестам. (csrc.nist.gov)

Основные методы получения:

1. **Метод серединных квадратов** Берётся число, возводится в квадрат, из середины результата выбираются цифры. Сейчас почти не используется из-за плохого качества.
2. **Линейный конгруэнтный метод**

$$X_{n+1} = (aX_n + c) \bmod m$$

$$U_n = \frac{X_n}{m}$$

где m — модуль, a — множитель, c — приращение, X_0 — seed.

3. **Мультипликативный конгруэнтный метод** Частный случай:

$$X_{n+1} = aX_n \bmod m$$

4. **Комбинированные генераторы** Объединяют несколько генераторов, чтобы увеличить период и улучшить статистические свойства.
5. **Современные генераторы** Например, Mersenne Twister, PCG, Xorshift, криптографические генераторы.
-

5. Метод обратной функции: идея, пример, дискретный случай

Основная идея метода обратной функции:

1. Генерируем $U \sim U(0, 1)$.
2. Берём функцию распределения нужной случайной величины $F(x)$.
3. Находим:

$$X = F^{-1}(U)$$

Тогда X имеет нужное распределение. Метод особенно удобен, если функция распределения известна, непрерывна, строго возрастает и её обратная функция легко находится. (bookdown.org)

Пример: экспоненциальное распределение.

$$F(x) = 1 - e^{-\lambda x}, \quad x \geq 0$$

Приравняем:

$$1 - e^{-\lambda x} = U$$

$$e^{-\lambda x} = 1 - U$$

$$-\lambda x = \ln(1 - U)$$

$$X = -\frac{1}{\lambda} \ln(1 - U)$$

Так как $(1-U)$ тоже равномерно распределено на $((0,1))$, часто пишут:

$$X = -\frac{1}{\lambda} \ln U$$

Особенность для дискретных распределений: обратная функция не является обычной гладкой функцией. Нужно построить накопленные вероятности и выбрать минимальное значение x_i , для которого:

$$F(x_i) \geq U$$

Например, если:

x_i	1	2	3	4
p_i	0.4	0.3	0.2	0.1
$F(x_i)$	0.4	0.7	0.9	1.0

то если $U = 0.65$, выбираем $X = 2$. Для дискретных распределений метод фактически сводится к поиску интервала накопленной вероятности. (rossetti.github.io)

6. Достоинства и недостатки метода обратной функции. Усечённое распределение

Достоинства:

- простой и понятный метод;
- даёт точное распределение, если F^{-1} известна;
- на каждое случайное число U получается одно значение X ;
- удобно использовать для равномерного, экспоненциального, Вейбулла и некоторых других распределений;
- хорошо подходит для дискретных распределений через накопленные вероятности.

Недостатки:

- не всегда существует простая обратная функция;
- для нормального распределения обратная функция не выражается простой элементарной формулой;
- для сложных распределений приходится использовать численные методы;
- для дискретных распределений нужен поиск по накопленным вероятностям.

Отдельно подчёркивается, что если CDF можно инвертировать, метод легко применяется; если аналитической формулы нет, приходится использовать численное обращение функции распределения. (rossetti.github.io)

Усечённое распределение.

Если исходная случайная величина имеет функцию распределения $F(x)$, но нужно получить значения только на интервале $([a,b])$, то используется формула:

$$X = F^{-1}(F(a) + U(F(b) - F(a)))$$

где:

- a — нижняя граница усечения;
- b — верхняя граница усечения;
- $U \sim U(0, 1)$;
- $F(a), F(b)$ — значения исходной функции распределения.

Идея простая: мы берём не весь интервал вероятностей $([0,1])$, а только часть $([F(a), F(b)])$.

7. Метод композиции для сложных законов распределения

Метод композиции используется, когда сложное распределение можно представить как смесь нескольких простых распределений.

Пусть плотность имеет вид:

$$f(x) = p_1 f_1(x) + p_2 f_2(x) + \dots + p_k f_k(x)$$

где:

$$p_i \geq 0, \quad \sum_{i=1}^k p_i = 1$$

Тогда алгоритм такой:

1. Сначала случайно выбираем компоненту i с вероятностью p_i .
2. Затем генерируем случайную величину из распределения $f_i(x)$.
3. Полученное значение и будет значением сложного распределения.

Пример: поток клиентов может состоять из двух типов:

- обычные клиенты — 80%;
- VIP-клиенты — 20%.

Тогда можно сначала выбрать тип клиента, а потом для каждого типа использовать своё распределение времени обслуживания.

Метод полезен для **смесей распределений**, эмпирических распределений, гиперэкспоненциальных распределений и случаев, когда поведение системы состоит из нескольких режимов. В современных учебниках по simulation random variate generation метод смеси/mixture distributions указывается как один из базовых способов генерации случайных величин наряду с inverse transform, convolution и acceptance-rejection. (rossetti.github.io)

8. Метод принятия-отклонения

Метод принятия-отклонения применяется, когда сложно напрямую сгенерировать величину из нужного распределения.

Пусть есть целевая плотность $f(x)$, из которой нужно генерировать значения. Выбираем более простую плотность $g(x)$, из которой легко генерировать, и константу M , такую что:

$$f(x) \leq M g(x)$$

Алгоритм:

1. Сгенерировать $Y \sim g(x)$.
2. Сгенерировать $U \sim U(0, 1)$.
3. Принять Y , если:

$$U \leq \frac{f(Y)}{M g(Y)}$$

4. Если условие не выполнено — отклонить Y и повторить.

Смысл: мы генерируем кандидатов из простого распределения, а затем оставляем только те, которые “похожи” на нужное распределение.

Плюс метода: можно генерировать значения из распределений, где нет удобной обратной функции. Минус: часть значений отбрасывается, поэтому метод может быть неэффективным. Эффективность зависит от того, насколько хорошо $Mg(x)$ “накрывает” $f(x)$; в высоких размерностях подобрать хорошее $g(x)$ становится трудно. (hpaulkeeler.com)

9. Способы генерирования нормального закона распределения

Нужно получить:

$$X \sim N(\mu, \sigma^2)$$

Обычно сначала получают стандартную нормальную величину:

$$Z \sim N(0, 1)$$

а потом переходят к нужным параметрам:

$$X = \mu + \sigma Z$$

1. Метод Бокса — Мюллера

Берём два независимых равномерных числа:

$$U_1, U_2 \sim U(0, 1)$$

Тогда:

$$Z_1 = \sqrt{-2 \ln U_1} \cos(2\pi U_2)$$

$$Z_2 = \sqrt{-2 \ln U_1} \sin(2\pi U_2)$$

Получаются две независимые стандартные нормальные величины. Этот метод прямо преобразует две равномерные случайные величины в две нормальные. (hpaulkeeler.com)

2. Полярный метод Марсальи

Генерируем:

$$V_1, V_2 \sim U(-1, 1)$$

Считаем:

$$S = V_1^2 + V_2^2$$

Если $S = 0$ или $S \geq 1$, пару отклоняем. Если $0 < S < 1$, то:

$$Z_1 = V_1 \sqrt{\frac{-2 \ln S}{S}}$$

$$Z_2 = V_2 \sqrt{\frac{-2 \ln S}{S}}$$

Преимущество: не нужны $\sin$ и $\cos$, но есть отклонение части пар.

3. Метод центральной предельной теоремы

Если сложить много равномерных случайных величин, сумма будет приближённо нормальной:

$$Z \approx \frac{\sum_{i=1}^n U_i - \frac{n}{2}}{\sqrt{n/12}}$$

Часто для $n = 12$:

$$Z \approx \sum_{i=1}^{12} U_i - 6$$

Плюс: очень простой метод. Минус: это только приближение, особенно плохо описывает хвосты нормального распределения.

4. Метод обратной функции

$$Z = \Phi^{-1}(U)$$

где Φ^{-1} — обратная функция распределения стандартного нормального закона. Проблема в том, что она не выражается простой элементарной формулой, поэтому обычно используются численные аппроксимации.

10. Тестирование генераторов случайных чисел на равномерность заполнения многомерного пространства

Одномерная проверка показывает только то, равномерно ли числа распределены на $([0,1])$. Но для имитационного моделирования этого недостаточно: последовательные числа могут иметь скрытые зависимости.

Например, генератор может давать хорошие значения по отдельности, но пары или тройки чисел могут ложиться на линии, плоскости или образовывать кластеры. Поэтому проверяют не только отдельные U_i , но и векторы:

$$(U_1, U_2), \quad (U_1, U_2, U_3), \quad \dots$$

или:

$$(U_i, U_{i+1}, \dots, U_{i+d-1})$$

Цель тестирования: убедиться, что точки равномерно заполняют d -мерный единичный куб:

$$[0, 1]^d$$

Один из подходов — **серийный тест**. Пространство $([0, 1]^d)$ делят на одинаковые ячейки, затем считают, сколько точек попало в каждую ячейку, и сравнивают с ожидаемым равномерным заполнением. В описании serial tests указывается, что такие тесты разбивают t -мерный гиперкуб на равные ячейки, генерируют точки-векторы и анализируют счётчики попаданий в ячейки. (epubs.siam.org)

Часто используют:

- визуальную проверку scatter plot для 2D и 3D;
- χ^2 -критерий;
- критерий Колмогорова — Смирнова для одномерной равномерности;
- тесты автокорреляции;
- серийные тесты;
- спектральный тест, особенно для линейных конгруэнтных генераторов;
- наборы тестов вроде NIST STS или TestU01.

Итог: хороший генератор должен давать не только равномерные отдельные числа, но и независимые последовательности, которые равномерно заполняют многомерное пространство. Это важно, потому что в имитационной модели несколько случайных чисел подряд часто используются для связанных событий: прихода клиента, выбора маршрута, времени обслуживания, вероятности отказа и т. д.

11. Тестирование генераторов случайных чисел на независимость

При тестировании генератора проверяют два главных свойства: **равномерность** и **независимость**. Равномерность означает, что числа хорошо покрывают $([0; 1])$, а независимость — что следующее число не должно предсказуемо зависеть от предыдущих. В учебных материалах по тестам RNG отдельно выделяются тесты независимости: **runs test**, **autocorrelation test**, **gap test**, **poker test** и др. (eg.bucknell.edu)

Основные способы проверки независимости:

1. Тест серий / runs test. Смотрят, как часто последовательность возрастает, убывает или пересекает некоторый уровень. Если серий слишком мало — числа “слипаются”; если слишком много — есть искусственное чередование.

Пример: берём уровень 0.5 и кодируем:

$$U_i > 0.5 \Rightarrow 1, \quad U_i < 0.5 \Rightarrow 0$$

Потом считаем количество серий одинаковых символов.

2. Тест автокорреляции. Проверяет связь между числами через лаг k :

$$r_k = \frac{\sum_{i=1}^{n-k} (U_i - \bar{U})(U_{i+k} - \bar{U})}{\sum_{i=1}^n (U_i - \bar{U})^2}$$

Если генератор хороший, то для $k > 0$:

$$r_k \approx 0$$

То есть U_i не должно быть связано с U_{i+1} , U_{i+2} и так далее.

3. Серийный тест. Последовательные числа объединяют в пары или тройки:

$$(U_1, U_2), (U_3, U_4), \dots$$

и проверяют, равномерно ли они заполняют квадрат или куб. Если точки ложатся на линии или образуют кластеры, независимость плохая.

4. Gap test. Проверяет длины промежутков между попаданиями чисел в заданный интервал. Например, считаем, сколько чисел проходит между попаданиями в $[0.2; 0.4]$.

5. Poker test. Числа группируют в блоки и смотрят повторяемость “комбинаций”, как в покере. Идея: если слишком часто появляются одинаковые структуры, значит есть зависимость.

В современных наборах тестов, например NIST Statistical Test Suite, используются frequency, runs, serial, approximate entropy и другие тесты для проверки статистической случайности последовательностей. (nvlpubs.nist.gov)

12. Построение моделей систем массового обслуживания. Примеры и части СМО

Система массового обслуживания, СМО — это система, в которую поступают заявки, они ожидают в очереди и затем обслуживаются каналами обслуживания.

Примеры СМО:

- касса в магазине;
- банк с операционистами;
- call-центр;
- поликлиника;
- аэропорт;
- сервер, обрабатывающий запросы;
- ремонтная служба.

Обычно СМО состоит из следующих частей:

1. **Источник заявок** Откуда приходят клиенты, заказы, звонки, машины, запросы.
2. **Входящий поток заявок** Описывает, как заявки поступают во времени. Часто используется пуассоновский поток.
3. **Очередь** Место, где заявки ждут обслуживания.
4. **Дисциплина очереди** Правило выбора заявки из очереди:

$$FIFO, LIFO, SIRO, Priority$$

Например, FIFO — первым пришёл, первым обслужен.

5. **Каналы обслуживания** Кассиры, операторы, серверы, станки, менеджеры.

6. **Процесс обслуживания** Время обслуживания может быть постоянным или случайным.
7. **Выход из системы** После обслуживания заявка покидает систему; иногда возможен отказ, уход из очереди или повторное обращение.

Для краткой записи моделей СМО часто используют обозначение Кендалла:

$$A/S/c/K/N/D$$

где A — распределение межприходных интервалов, S — распределение времени обслуживания, c — число каналов, K — вместимость системы, N — размер источника заявок, D — дисциплина обслуживания. Например, $M/M/1$ означает пуассоновский входящий поток, экспоненциальное обслуживание и один канал. ([Википедия](#))

13. Основные критерии оценки работы СМО

Основные характеристики СМО показывают, насколько система загружена, насколько велики очереди и сколько времени заявки проводят в системе.

Главные показатели:

$$\lambda$$

— интенсивность поступления заявок, то есть среднее число заявок в единицу времени.

$$\mu$$

— интенсивность обслуживания одним каналом, то есть среднее число заявок, которое канал может обслужить за единицу времени.

$$\rho$$

— коэффициент загрузки системы. Для одноканальной СМО:

$$\rho = \frac{\lambda}{\mu}$$

Для многоканальной системы с c каналами:

$$\rho = \frac{\lambda}{c\mu}$$

Если ρ близко к 1, система почти полностью загружена, очереди могут резко расти.

Основные критерии:

Обозначение	Смысл
L	среднее число заявок в системе
L_q	среднее число заявок в очереди

Обозначение	Смысл
W	среднее время пребывания заявки в системе
W_q	среднее время ожидания в очереди
P_0	вероятность простоя системы
P_n	вероятность того, что в системе n заявок
$P_{\text{отк}}$	вероятность отказа, если система имеет ограниченную вместимость
Q	относительная пропускная способность
A	абсолютная пропускная способность
ρ	загрузка каналов обслуживания

Важная связь — **формула Литтла**:

$$L = \lambda W$$

Она означает: среднее число заявок в системе равно интенсивности входа, умноженной на среднее время пребывания заявки в системе. Аналогично для очереди:

$$L_q = \lambda W_q$$

Закон Литтла широко используется для анализа очередей и систем обслуживания. ([Википедия](#))

14. Моделирование поступления заявок на основе стационарного пуассоновского процесса

Стационарный пуассоновский процесс используется, когда интенсивность поступления заявок постоянна:

$$\lambda = \text{const}$$

Тогда число заявок за интервал времени t имеет распределение Пуассона:

$$P\{N(t) = k\} = \frac{(\lambda t)^k}{k!} e^{-\lambda t}$$

где:

- $N(t)$ — число заявок за время t ;
- λ — средняя интенсивность поступления;
- k — число заявок.

Межприходные интервалы имеют экспоненциальное распределение:

$$\tau_i \sim \text{Exp}(\lambda)$$

Для моделирования можно использовать формулу:

$$\tau_i = -\frac{1}{\lambda} \ln U_i$$

где $U_i \sim U(0, 1)$.

Моменты поступления заявок:

$$T_1 = \tau_1$$

$$T_2 = \tau_1 + \tau_2$$

$$T_n = \sum_{i=1}^n \tau_i$$

Классические свойства пуассоновского потока:

1. **Стационарность** Вероятность поступления заявок зависит только от длины интервала, а не от его положения во времени.
2. **Отсутствие последействия** Будущие поступления не зависят от прошлых.
3. **Ординарность** Вероятность одновременного появления двух и более заявок за очень малый интервал мала.
4. **Независимые приращения** Число событий на непересекающихся интервалах независимо.

В источниках по пуассоновским процессам это формулируется как свойства стационарных и независимых приращений, а межприходные интервалы описываются независимыми экспоненциальными величинами. ([MIT OpenCourseWare](#))

15. Моделирование нестационарного пуассоновского процесса

В нестационарном пуассоновском процессе интенсивность зависит от времени:

$$\lambda = \lambda(t)$$

Например, в магазине утром клиентов мало, вечером много; в call-центре после рекламы поток звонков резко возрастает.

Число заявок на интервале $([a, b])$ имеет распределение Пуассона со средним:

$$\Lambda(a, b) = \int_a^b \lambda(t) dt$$

$$P\{N(a, b) = k\} = \frac{\Lambda(a, b)^k}{k!} e^{-\Lambda(a, b)}$$

Почему нужен другой алгоритм?

В стационарном процессе λ постоянно, поэтому межприходные интервалы независимы и одинаково распределены:

$$\tau_i \sim \text{Exp}(\lambda)$$

А в нестационарном процессе $\lambda(t)$ меняется, поэтому нельзя просто генерировать одинаковые экспоненциальные интервалы. Вероятность прихода зависит от текущего времени.

Основные алгоритмы моделирования:

1. Метод прореживания / thinning

Выбирают верхнюю границу интенсивности:

$$\lambda(t) \leq \lambda^*$$

Дальше:

1. Генерируют кандидатов как стационарный пуассоновский поток с интенсивностью λ^* .
2. Для каждого момента t генерируют $U \sim U(0, 1)$.
3. Принимают заявку, если:

$$U \leq \frac{\lambda(t)}{\lambda^*}$$

Иначе заявку отклоняют.

Этот алгоритм действительно используется для моделирования nonhomogeneous Poisson process; в конспекте Columbia прямо приводится схема: генерировать кандидатов с λ^* , затем принимать событие с вероятностью $\lambda(t)/\lambda^*$. (columbia.edu)

2. Метод обратного преобразования интенсивности

Используют накопленную интенсивность:

$$\Lambda(t) = \int_0^t \lambda(s) ds$$

Сначала генерируют процесс с интенсивностью 1, а потом преобразуют время через обратную функцию Λ^{-1} . Такой подход также описывается как rate inversion algorithm для нестационарного пуассоновского процесса. (rossetti.github.io)

16. Отличие непрерывных моделей от дискретных. Способы решения непрерывных моделей

Дискретные модели описывают систему, состояние которой меняется скачками: по событиям или по шагам времени. Например:

- пришёл клиент;

- освободилась касса;
- произошла поломка;
- заявка покинула систему.

Такие модели часто применяются в системах массового обслуживания, логистике, бизнес-процессах.

Непрерывные модели описывают систему, состояние которой меняется плавно во времени. Обычно они задаются дифференциальными уравнениями:

$$\frac{dx}{dt} = f(t, x)$$

Например:

- изменение численности населения;
- изменение уровня запасов;
- рост капитала;
- распространение инфекции;
- изменение температуры.

Основные способы решения непрерывных моделей:

1. Аналитическое решение Если уравнение простое, можно найти точную формулу:

$$x(t) = \dots$$

Но для сложных систем это часто невозможно.

2. Численное решение Состояние системы считают по шагам времени. Основные методы:

Метод Эйлера:

$$x_{n+1} = x_n + hf(t_n, x_n)$$

Методы Рунге — Кутты, например RK4:

$$k_1 = f(t_n, x_n)$$

$$k_2 = f\left(t_n + \frac{h}{2}, x_n + \frac{h}{2}k_1\right)$$

$$k_3 = f\left(t_n + \frac{h}{2}, x_n + \frac{h}{2}k_2\right)$$

$$k_4 = f(t_n + h, x_n + hk_3)$$

$$x_{n+1} = x_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Учебные материалы по численному моделированию ОДУ обычно выделяют метод Эйлера, методы Рунге — Кутты, Adams-Bashforth, Adams-Moulton и backward differentiation formulas как основные классы численных методов. (wr.informatik.uni-hamburg.de)

17. Особенности непрерывных моделей. Примеры

Главная особенность непрерывных моделей — состояние системы задаётся функциями времени:

$$x(t), y(t), z(t)$$

и меняется непрерывно, а не только в моменты событий.

Обычно непрерывная модель имеет вид:

$$\frac{dx_1}{dt} = f_1(x_1, x_2, \dots, x_n, t)$$

$$\frac{dx_2}{dt} = f_2(x_1, x_2, \dots, x_n, t)$$

...

$$\frac{dx_n}{dt} = f_n(x_1, x_2, \dots, x_n, t)$$

Особенности:

- описывает плавную динамику;
- часто строится на дифференциальных уравнениях;
- требует задания начальных условий;
- позволяет анализировать равновесия, устойчивость, рост, спад, колебания;
- обычно решается численно, если система сложная.

Примеры непрерывных моделей:

1. Рост населения

$$\frac{dN}{dt} = rN$$

где N — численность населения, r — темп роста.

2. Логистический рост

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K}\right)$$

где K — предельная ёмкость среды.

3. Модель хищник — жертва

$$\frac{dx}{dt} = ax - bxy$$

$$\frac{dy}{dt} = -cy + dxy$$

где x — жертвы, y — хищники.

4. Модель запасов на складе

$$\frac{dS}{dt} = \text{Поступление} - \text{Расход}$$

5. Экономическая модель капитала

$$\frac{dK}{dt} = I - \delta K$$

где K — капитал, I — инвестиции, δ — износ.

Дифференциальные уравнения являются естественным способом описания динамически изменяющихся систем в физике, биологии, экономике и социальных науках. ([hps.vi4io.org](https://vi4io.org))

18. Особенности моделей системной динамики Дж. Форрестера

Системная динамика — подход Дж. Форрестера для моделирования сложных систем с обратными связями, накоплениями и задержками. В системной динамике система рассматривается не как отдельные события, а как совокупность потоков и накопителей.

Основные элементы:

1. Накопители / *stocks*

Это то, что накапливается:

Stock

Например:

- численность населения;
- запас товара;
- капитал;
- количество клиентов;
- объём загрязнения.

2. Потоки / *flows*

Это скорости изменения накопителей:

$$\frac{dStock}{dt} = Inflow - Outflow$$

Например:

- рождаемость и смертность;
- поступление и продажа товара;
- инвестиции и амортизация.

Форрестер подчёркивал, что системы можно описывать через два базовых типа понятий — **stocks** и **flows**. ([Gatech Sites](#))

3. Обратные связи

Связи, где результат влияет на причину.

Положительная обратная связь усиливает изменение:

Больше клиентов $\Rightarrow$ больше рекомендаций $\Rightarrow$ ещё больше клиентов

Отрицательная обратная связь стабилизирует систему:

Мало товара $\Rightarrow$ увеличить заказ $\Rightarrow$ запас восстановился

4. Задержки

Решение может влиять на систему не сразу. Например, заказ товара сделан сегодня, а поставка пришла через неделю.

5. Агрегированный уровень описания

В системной динамике обычно не моделируют каждого человека отдельно. Вместо этого моделируют группы и потоки: население, спрос, запасы, деньги, производство.

В AnyLogic системная динамика также описывается как подход, где используются дифференциальные уравнения, численные методы Эйлера и Рунге — Кутты; такие модели часто детерминированы, если специально не добавлять случайность. ([anylogic.com](#))

19. Особенность агентно-ориентированных моделей и классификация среды

Агентно-ориентированное моделирование строит систему “снизу вверх”: сначала задаются отдельные агенты, их поведение и взаимодействия, а затем из этих локальных действий возникает поведение всей системы.

Главная особенность:

микроуровень $\Rightarrow$ макроэффект

То есть общий результат возникает из поведения отдельных агентов. Например, пробка возникает не потому, что кто-то “задал пробку”, а потому что многие водители принимают локальные решения. AnyLogic определяет агентное моделирование как децентрализованный, индивидуально-ориентированный подход к построению модели. ([anylogic.help](#))

Примеры агентов:

- покупатели;

- автомобили;
- фирмы;
- банки;
- животные;
- люди в толпе;
- роботы;
- пользователи социальной сети.

Среда — это пространство или условия, в которых агенты существуют и взаимодействуют.

Классификация среды:

Вид среды	Смысл	Пример
Дискретная	пространство разделено на клетки или узлы	клеточный автомат, шахматная доска
Непрерывная	координаты могут быть любыми	движение людей в торговом центре
Сетевая	агенты связаны графом	социальная сеть, транспортная сеть
Без явного пространства	важны только состояния и связи	рынок фирм
Статическая	среда сама почти не меняется	карта дорог без изменений
Динамическая	среда меняется во времени	пробки, погода, цены
Детерминированная	одинаковые действия дают одинаковый результат	фиксированные правила движения
Стохастическая	присутствует случайность	случайный выбор маршрута
Полностью наблюдаемая	агент знает всю среду	простая учебная модель
Частично наблюдаемая	агент видит только часть среды	водитель видит только ближайшие машины

В агентных моделях агенты взаимодействуют не только друг с другом, но и со средой; разработчик должен задать агентов, отношения, топологию взаимодействий и среду. (informs-sim.org)

20. Особенность агентно-ориентированных моделей и классификация агентов

Главная особенность агентно-ориентированной модели — система состоит из **автономных агентов**. Каждый агент имеет своё состояние, правила поведения, память, цели и может взаимодействовать с другими агентами.

В AnyLogic агент описывается как основной строительный блок модели: он может иметь поведение, память, события, диаграммы состояний, контакты и вложенные объекты. (anylogic.help)

Классификация агентов:

Вид агентов	Смысл	Пример
Пассивные	сами почти не принимают решений	товар, документ, заявка
Активные	имеют собственное поведение	клиент, автомобиль, фирма
Реактивные	реагируют на ситуацию по простым правилам	человек уходит, если очередь длинная
Когнитивные / интеллектуальные	имеют цели, память, обучение, выбор стратегии	компания выбирает цену
Однородные	все агенты одного типа	одинаковые покупатели
Неоднородные	агенты различаются параметрами	обычные и VIP-клиенты
Мобильные	перемещаются в среде	пешеходы, машины
Неподвижные	имеют фиксированное положение	магазины, станции, кассы
Индивидуальные	представляют один объект	один человек
Групповые	представляют группу объектов	домохозяйство, фирма, команда
Обучающиеся	меняют поведение по опыту	инвестор меняет стратегию
Необучающиеся	действуют по фиксированным правилам	автоматический станок

Типичный агент имеет:

Состояние + Правила + Цель + Взаимодействия

Например, агент “покупатель” может иметь параметры:

доход, возраст, терпение, список покупок, скорость

и правила:

- войти в магазин;
- выбрать товар;
- встать в очередь;
- уйти, если ожидание слишком долгое;
- оплатить покупку.

В литературе по АВМ подчёркивается, что агентные модели подходят для сложных систем, где автономные агенты взаимодействуют, влияют друг на друга, адаптируются, а общий порядок возникает из локальных правил. ([Springer](#))

21. Основная особенность агентно-ориентированных моделей, примеры

Агентно-ориентированная модель строится “снизу вверх”: сначала задаются отдельные агенты, их свойства, правила поведения и взаимодействия, а уже потом из поведения множества агентов получается общая динамика системы. AnyLogic описывает

агентное моделирование как подход, где фокус находится на отдельных объектах, их поведении и взаимодействии. (anylogic.com)

Главная идея:

поведение отдельных агентов $\Rightarrow$ поведение всей системы

Агент может иметь:

состояние + правила + цели + память + взаимодействия

Примеры:

- покупатели в магазине;
- автомобили на дороге;
- пассажиры в аэропорту;
- фирмы на рынке;
- банки и клиенты;
- люди при эвакуации;
- животные в экосистеме;
- пользователи социальной сети.

Например, в модели магазина каждый покупатель сам решает: прийти, выбрать товар, встать в очередь, оплатить или уйти. А общие показатели — длина очереди, прибыль, загруженность касс — возникают из действий всех покупателей.

22. Способы продвижения модельного времени

В имитационном моделировании есть несколько способов продвижения времени.

1. Фиксированный шаг времени

Модельное время увеличивается на постоянный шаг:

$$t_{n+1} = t_n + \Delta t$$

Например, каждые 1 секунду, 1 минуту или 1 час пересчитывается состояние системы.

Применяется в:

- непрерывных моделях;
- системной динамике;
- моделях с дифференциальными уравнениями;
- моделях запасов, популяций, эпидемий.

Плюс: просто реализовать. Минус: если шаг слишком большой — будет неточность, если слишком маленький — модель работает медленно.

2. Событийное продвижение времени / next-event time advance

Время “перепрыгивает” сразу к ближайшему событию:

$$t \rightarrow t_{\text{следующего события}}$$

Например:

- пришёл клиент;
- освободилась касса;
- сломался станок;
- закончилась обработка заявки.

Применяется в:

- дискретно-событийных моделях;
- системах массового обслуживания;
- логистике;
- производстве;
- моделях банков, магазинов, call-центров.

В AnyLogic дискретно-событийная модель выполняется через события, а модельные часы могут мгновенно переходить к следующему событию. (anylogic.com)

3. Комбинированный / гибридный способ

Используются и шаги времени, и события. Например, в AnyLogic модель может выполняться как последовательность **time steps** и **event steps**. (anylogic.help)

Применяется в гибридных моделях, где есть:

- непрерывная динамика, например уровень запасов;
 - дискретные события, например приход клиента;
 - агенты, которые перемещаются и принимают решения.
-

23. Этапы исследования систем с помощью имитационного моделирования

Типовая схема имитационного исследования включает несколько этапов. В учебниках по simulation methodology обычно выделяют: постановку задачи, построение концептуальной модели, сбор данных, программную реализацию, верификацию, валидацию, эксперименты и анализ результатов. (rossetti.github.io)

Этапы:

1. **Постановка проблемы** Нужно понять, какую систему исследуем и зачем.
2. **Формулировка цели и критериев** Например: уменьшить очередь, повысить прибыль, снизить время ожидания.
3. **Описание системы** Определяются элементы системы, входы, выходы, ограничения.
4. **Построение концептуальной модели** Описывается логика системы без программирования: схема, блоки, правила.
5. **Сбор и анализ исходных данных** Например: поток клиентов, время обслуживания, расписание, вероятности.
6. **Выбор распределений и параметров** Например:

$$\tau \sim Exp(\lambda)$$

для времени между поступлениями заявок.

7. **Программная реализация модели** Модель создаётся в AnyLogic, Python, Arena, GPSS и т. д.
8. **Верификация** Проверка: правильно ли модель запрограммирована.
9. **Валидация** Проверка: похожа ли модель на реальную систему.
10. **Планирование экспериментов** Определяются факторы, уровни факторов, число прогонов.
11. **Проведение экспериментов** Запускаются сценарии и собираются результаты.
12. **Анализ результатов** Рассчитываются средние значения, доверительные интервалы, сравнение вариантов.
13. **Выводы и рекомендации** Например: сколько кассиров нужно, какой режим лучше, где узкое место.

24. Какие модели называются адекватными? Валидация и верификация

Адекватная модель — это модель, которая с достаточной для цели исследования точностью отражает реальную систему. Важно: модель не обязана быть полной копией реальности. Она должна быть достаточно точной именно для поставленной задачи.

Например, если мы моделируем очередь в магазине, нам не нужно описывать настроение каждого покупателя, если цель — оценить среднее время ожидания.

Верификация отвечает на вопрос:

Правильно ли мы построили модель?

То есть нет ли ошибок в коде, формулах, логике, связях блоков.

Валидация отвечает на вопрос:

Правильную ли модель мы построили?

То есть соответствует ли модель реальной системе. В работах Сарджента по verification and validation подчёркивается, что V&V нужны для определения, достаточно ли корректно модель представляет систему для целей исследования. ([informs-sim.org](https://www.informs-sim.org))

Схема обеспечения адекватности:

```

Реальная система
    ↓ наблюдение, сбор данных
Концептуальная модель
    ↓ проверка логики с экспертами
Программная модель
    ↓ верификация
Результаты модели
    ↓ сравнение с реальными данными
Вывод об адекватности
  
```

Если результаты модели близки к реальным данным в пределах допустимой ошибки, модель считают адекватной.

25. Схема сравнения выходных данных реальной системы и модели

Адекватность часто проверяют сравнением выходов реальной системы и модели.

Пусть есть реальный показатель:

$$Y_{\text{real}}$$

и модельный показатель:

$$Y_{\text{model}}$$

Тогда ошибка:

$$\varepsilon = |Y_{\text{real}} - Y_{\text{model}}|$$

или относительная ошибка:

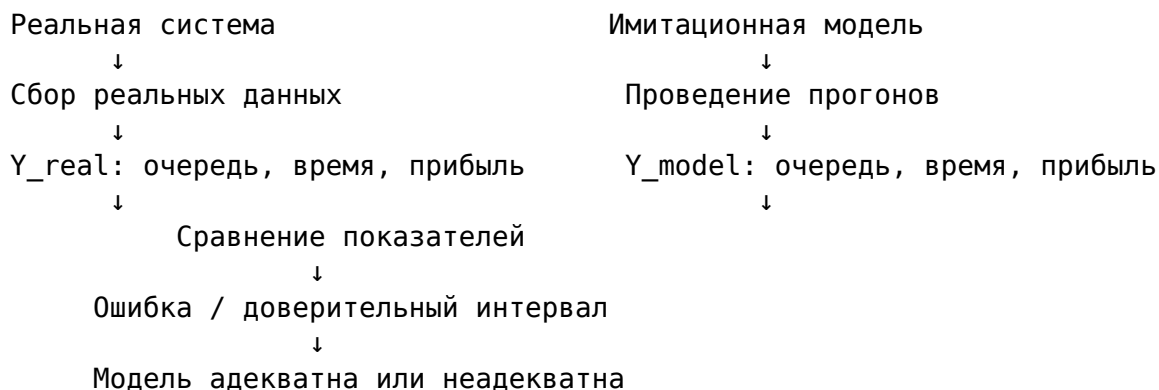
$$\delta = \frac{|Y_{\text{real}} - Y_{\text{model}}|}{Y_{\text{real}}} \cdot 100\%$$

Если:

$$\delta \leq \delta_{\text{доп}}$$

то модель можно считать адекватной по этому показателю.

Схема сравнения:



Сравнивать можно:

- среднее время ожидания;
- среднюю длину очереди;
- загрузку оборудования;
- число обслуженных заявок;
- прибыль;
- вероятность отказа;

- распределение выходных значений.

Важно сравнивать не только средние значения, но и разброс:

$$\bar{Y}_{\text{real}} \approx \bar{Y}_{\text{model}}$$

$$\sigma_{\text{real}} \approx \sigma_{\text{model}}$$

26. Планирование экспериментов на имитационных моделях. Факторы и отклики

Планирование экспериментов нужно, чтобы не запускать модель хаотично, а заранее определить, какие параметры менять, какие результаты измерять и сколько прогонов делать.

Главные цели:

- понять, какие параметры сильнее всего влияют на результат;
- сравнить альтернативные варианты системы;
- найти оптимальные настройки;
- уменьшить число лишних экспериментов;
- получить статистически надёжные выводы.

В терминах design of experiments входные параметры и структурные предположения модели называются **факторами**, а выходные показатели — **откликами**. Факторы могут быть количественными и качественными. (informatics-sim.org)

Факторы — то, что мы изменяем.

Виды факторов:

Вид фактора	Пример
Количественные	число кассиров, интенсивность потока, время обслуживания
Качественные	дисциплина очереди FIFO/priority, тип расписания
Управляемые	количество сотрудников, режим работы
Неуправляемые	случайный спрос, погодные условия
Постоянные	фиксированные условия эксперимента
Случайные	параметры, меняющиеся случайно

Отклики — то, что мы измеряем.

Примеры откликов:

$$W_q$$

— среднее время ожидания в очереди;

$$L_q$$

— средняя длина очереди;

$$\rho$$

— загрузка оборудования или персонала;

$$P_{\text{отк}}$$

— вероятность отказа;

$$Profit$$

— прибыль;

$$Cost$$

— затраты.

27. Проблема сравнения альтернативных конфигураций. Метод общих случайных чисел

Главная проблема при сравнении альтернатив: результаты имитационной модели случайны.

Например, мы сравниваем две конфигурации магазина:

- вариант А: 1 кассир;
- вариант В: 2 кассира.

Если в первом прогоне в варианте А случайно пришло мало клиентов, а в варианте В случайно пришло много клиентов, сравнение будет нечестным. Разница может возникнуть не из-за конфигурации, а из-за случайности.

Пусть результат альтернативы А:

$$Y_A$$

результат альтернативы В:

$$Y_B$$

Нас интересует:

$$D = Y_A - Y_B$$

Проблема — большая дисперсия оценки:

$$Var(D)$$

Метод общих случайных чисел / Common Random Numbers, CRN — это приём, при котором разные альтернативы моделируются на одинаковых случайных входах. Например, один и тот же поток клиентов используется для варианта А и варианта В.

Идея:

Одинаковые случайные условия

↓

Альтернатива А

Альтернатива В

↓

Сравнение становится честнее

CRN используется как метод снижения дисперсии при сравнении стохастических систем. В источниках подчёркивается, что метод синхронизирует случайные числа между прогонами, чтобы индуцировать корреляцию выходов и легче увидеть различия между альтернативами. (PMC)

Формально:

$$Var(Y_A - Y_B) = Var(Y_A) + Var(Y_B) - 2Cov(Y_A, Y_B)$$

Если с помощью общих случайных чисел сделать ковариацию положительной:

$$Cov(Y_A, Y_B) > 0$$

то дисперсия разности уменьшается, и сравнение становится точнее.

28. Проблемы синхронизации случайных чисел при сравнении альтернатив

Главная проблема та же: нужно сравнивать альтернативы в одинаковых случайных условиях. Но на практике трудно добиться правильной синхронизации случайных чисел.

Синхронизация означает: одно и то же случайное число должно использоваться для одной и той же цели во всех альтернативных моделях. Например, случайное число для времени прихода 10-го клиента должно использоваться именно для 10-го клиента в обеих конфигурациях.

Основные проблемы:

1. Разное количество случайных событий

В одной конфигурации заявка может обслужиться, а в другой — уйти из очереди. Тогда одна модель использует больше случайных чисел, чем другая, и последовательности “съезжают”.

2. Разная логика альтернатив

Например, в варианте А есть один кассир, а в варианте В — два. Из-за этого порядок обслуживания меняется, а значит, случайные числа могут начать использоваться для разных клиентов.

3. Переменное число агентов

В агентных моделях агенты могут появляться, исчезать, менять маршруты. Это усложняет сохранение одинаковых случайных условий.

4. Использование одного потока для разных целей

Плохая практика — брать все случайные числа из одного потока: и для прихода клиентов, и для времени обслуживания, и для выбора маршрута. Тогда изменение одной части модели ломает всю синхронизацию.

5. Параллельные вычисления

Если модель запускается параллельно, порядок использования случайных чисел может меняться, и воспроизводимость нарушается.

6. Разные длительности прогонов

Если одна альтернатива завершает прогон раньше или позже, количество использованных случайных чисел различается.

Чтобы решать эти проблемы, используют **отдельные потоки случайных чисел**: один поток для приходов, другой — для обслуживания, третий — для выбора маршрута. В работах по random number streams подчёркивается, что потоки и подпотоки нужны для синхронизации CRN и воспроизводимости экспериментов. ([The R Journal](#))

Пример правильного разделения:

Stream 1 — приходы клиентов

Stream 2 — время обслуживания

Stream 3 — выбор типа клиента

Stream 4 — решение уйти или ждать

29. Алгоритм выбора лучшей конфигурации из множества альтернатив

Когда альтернатив много, нельзя просто сделать по одному прогону и выбрать лучший результат. Из-за случайности можно выбрать не действительно лучшую систему, а ту, которой случайно повезло.

Задача называется **ranking and selection**: нужно выбрать альтернативу с наилучшим средним показателем. В литературе по indifference-zone ranking and selection цель формулируется как выбор лучшей альтернативы среди конечного множества симулируемых вариантов с заданной вероятностью правильного выбора. (people.orie.cornell.edu)

Пусть есть k альтернатив:

$$A_1, A_2, \dots, A_k$$

и для каждой есть математическое ожидание результата:

$$\mu_i = E[Y_i]$$

Если нужно минимизировать показатель, например время ожидания, лучшая альтернатива:

$$A^* = \arg \min_i \mu_i$$

Если максимизировать, например прибыль:

$$A^* = \arg \max_i \mu_i$$

Простой алгоритм выбора лучшей конфигурации:

1. Задать альтернативы

$$A_1, A_2, \dots, A_k$$

Например: 1 кассир, 2 кассира, 3 кассира.

2. Выбрать критерий качества

Например:

$$W_q \rightarrow \min$$

или:

$$Profit \rightarrow \max$$

3. Задать число начальных прогонов

$$n_0$$

Например, по 10 прогонов для каждой альтернативы.

4. Запустить каждую альтернативу n_0 раз

Получаем результаты:

$$Y_{ij}$$

где i — номер альтернативы, j — номер прогона.

5. Рассчитать среднее значение

$$\bar{Y}_i = \frac{1}{n} \sum_{j=1}^n Y_{ij}$$

6. Рассчитать дисперсию

$$S_i^2 = \frac{1}{n-1} \sum_{j=1}^n (Y_{ij} - \bar{Y}_i)^2$$

7. Построить доверительный интервал

$$\bar{Y}_i \pm t_{\alpha/2, n-1} \frac{S_i}{\sqrt{n}}$$

8. Исключить явно худшие альтернативы

Если доверительный интервал одной альтернативы хуже другой, её можно убрать.

9. Добавить прогоны для близких альтернатив

Если результаты близкие, нужно больше повторений.

10. Выбрать лучшую альтернативу

Для минимизации:

$$A^* = \arg \min_i \bar{Y}_i$$

Для максимизации:

$$A^* = \arg \max_i \bar{Y}_i$$

11. Проверить устойчивость результата

Проверить, сохраняется ли выбор при других seed, параметрах и сценариях.

30. Основная идея метода Монте-Карло. Примеры

Метод Монте-Карло — это метод решения задач с помощью многократного случайного моделирования. Вместо того чтобы искать точное аналитическое решение, мы много раз генерируем случайные варианты и оцениваем результат статистически. IBM определяет Monte Carlo simulation как вычислительный алгоритм, использующий повторную случайную выборку для оценки вероятности диапазона возможных результатов. (ibm.com)

Общая схема:

Задать модель

↓

Задать случайные входные параметры

↓

Много раз провести случайные испытания

↓

Получить распределение результатов

↓

Оценить среднее, риск, вероятность, доверительный интервал

Если нужно оценить математическое ожидание:

$$E[X]$$

генерируем n случайных значений:

$$X_1, X_2, \dots, X_n$$

и считаем:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

При большом n :

$$\bar{X} \approx E[X]$$

Пример 1. Оценка числа π

Берём квадрат $([-1;1] \times [-1;1])$, внутри него круг радиуса 1.

Площадь квадрата:

$$S_{\text{квадрата}} = 4$$

Площадь круга:

$$S_{\text{круга}} = \pi$$

Доля точек, попавших в круг:

$$p \approx \frac{\pi}{4}$$

Тогда:

$$\pi \approx 4p$$

Если из N случайных точек M попало в круг:

$$\pi \approx 4 \frac{M}{N}$$

Пример 2. Оценка риска проекта

Пусть прибыль проекта зависит от случайного спроса, цены и затрат:

$$Profit = Revenue - Cost$$

Много раз генерируем спрос, цену и затраты, получаем распределение прибыли и оцениваем:

$$P(Profit < 0)$$

То есть вероятность убытка.

Пример 3. Очередь в магазине

Случайно моделируем:

- приход покупателей;
- время выбора товаров;
- время обслуживания;
- вероятность ухода из очереди.

После многих прогонов оцениваем:

$$W_q$$

— среднее ожидание в очереди;

$$L_q$$

— среднюю длину очереди;

$$\rho$$

— загрузку кассиров.

Главный смысл метода Монте-Карло: если система содержит случайность и её сложно решить точно, можно много раз проиграть случайные сценарии и получить приближённый, но полезный результат.